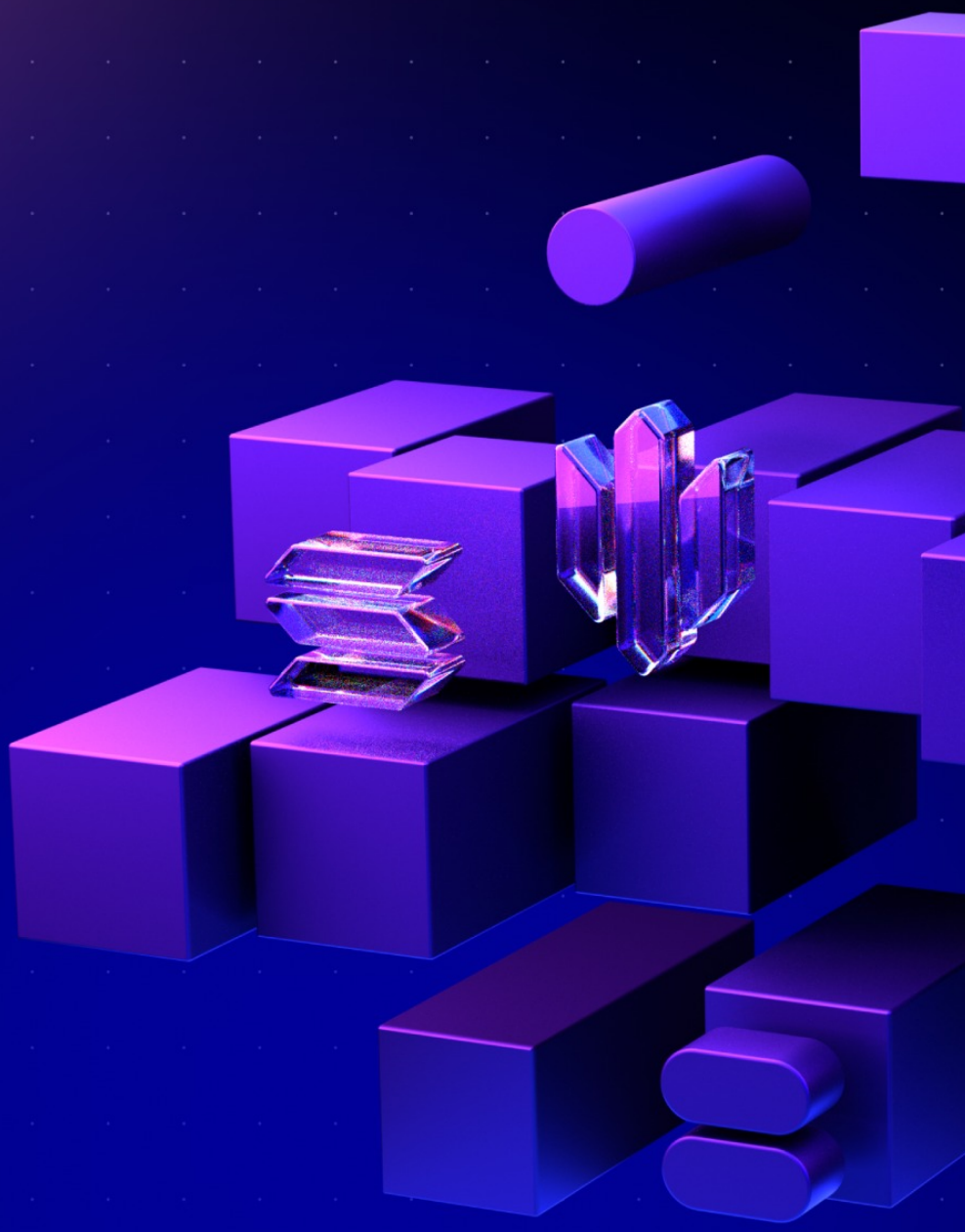
ackee
blockchain security

Kamino

Kamino Vaults

14.10.2025



Audit Report

Contents

1. Document Revisions . . . 3
2. Overview . . . 4
   2.1. Ackee Blockchain Security . . . 4
   2.2. Audit Methodology . . . 5
   2.3. Finding Classification . . . 6
   2.4. Review Team . . . 8
   2.5. Disclaimer . . . 8
3. Executive Summary . . . 9
   Revision 1.0 . . . 9
   Revision 1.1 . . . 11
4. Findings Summary . . . 13
Report Revision 1.0 . . . 14
   Revision Team . . . 14
   System Overview . . . 14
   Trust Model . . . 15
   Fuzzing . . . 16
   Findings . . . 17
Report Revision 1.1 . . . 27
   Revision Team . . . 27
Appendix A: How to cite . . . 28
Appendix B: Trident Findings . . . 29
   B.1. Fuzzing . . . 29

1. Document Revisions

1.0-draft	Draft Report	10.09.2025
1.1-draft	Fix Review	14.10.2025
1.1	Final Report	14.10.2025

2. Overview

This document presents our findings in reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Audit Methodology

The Ackee Blockchain Security auditing process follows a routine series of steps:

1. **Code review**
   a. High-level review of the specifications, sources, and instructions provided to us to make sure we understand the project's size, scope, and functionality.
   b. Detailed manual code review, which is the process of reading the source code line-by-line to identify potential vulnerabilities. We focus mainly on common classes of Solana program vulnerabilities, such as:

      missing ownership checks, missing signer authorization, signed CPI of unverified programs, cosplay of Solana accounts, missing rent exemption assertion, bump seed canonicalization, incorrect accounts closing, casting truncation, numerical precision errors, arithmetic overflows or underflows.

   c. Comparison of the code and given specifications, ensuring that the program logic correctly implements everything intended.
   d. Review of best practices to improve efficiency, clarity, and maintainability.
2. **Testing and automated analysis**
   a. Run client's tests to ensure that the system works as expected, potentially write missing unit or fuzzy tests using our testing framework Trident.
3. **Local deployment + hacking**
   a. The programs are deployed locally, and we try to attack the system and break it. There is no specific strategy here, and each project's attack attempts are unique to its implementation.

2.3. Finding Classification

A *Severity* rating of each finding is determined as a synthesis of two sub-ratings: *Impact* and *Likelihood*. It ranges from *Informational* to *Critical*.

If we have found a scenario in which an issue is exploitable, it will be assigned an impact rating of *High*, *Medium*, or *Low*, based on the direness of the consequences it has on the system. If we haven't found a way, or the issue is only exploitable given a change in *configuration* (system settings or parameters, such as deployment scripts, compiler configurations, using multi-signature wallets for owners, etc.) or given a change in the codebase, then it will be assigned an impact rating of *Warning* or *Info*.

Low to *High* impact issues also have a *Likelihood*, which measures the probability of exploitability during runtime.

The full definitions are as follows:

Severity

		Likelihood			
		High	**Medium**	**Low**	**N/A**
Impact	**High**	Critical	High	Medium	-
	Medium	High	Medium	Low	-
	Low	Medium	Low	Low	-
	Warning	-	-	-	Warning
	Info	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue cannot be exploited given the current code and/or `configuration`, but could be a security vulnerability if these were to change slightly. If we haven't found a way to exploit the issue given the time constraints, it might be marked as a "Warning" or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations. Another example is that the issue would be security-related if code or `configuration` was to change.

Likelihood

- **High** - The issue is exploitable by virtually anyone under virtually any circumstance.
- **Medium** - Exploiting the issue currently requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

The following table lists all contributors to this report. For authors of the specific revision, see the “Revision team” section in the respective “Report revision” chapter.

Member’s Name	Position
Andrej Lukačovič	Lead Auditor
Max Kupchenko	Auditor
Matej Hyčko	Auditor
Felipe Donato	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We’ve put our best effort to find all vulnerabilities in the system, however our findings shouldn’t be considered as a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

3. Executive Summary

Kamino Vaults is a yield aggregator protocol that optimizes capital allocation across Kamino Lending reserves. The protocol uses a share-based accounting model where users receive share tokens representing their proportional ownership of the vault's assets.

Revision 1.0

Kamino engaged Ackee Blockchain Security to perform a security review of Kamino Kamino Vaults with a total time donation of 19 engineering days in a period between August 13 and September 12, 2025, with Andrej Lukačovič as the lead auditor.

The audit was performed on the commit `0c5aa72`[1] in the contracts repository and the scope was the following:

- https://github.com/Kamino-Finance/kvault, excluding external dependencies.

We organized our security review into three phases to thoroughly cover the Kamino Vaults protocol.

We started by analyzing the Kamino Vaults protocol's design, architecture, and business logic. In this phase, we reviewed all available documentation, explored the frontend functionality, studied the intended use cases, and understood how the automated liquidity management works. We also examined how the protocol integrates with external systems and mapped out the trust model and operational workflows.

In the second phase, we performed a detailed manual review of the source code, wrote proof-of-concept exploits for vulnerabilities we identified, and built integration tests. We worked closely with other auditors to discuss attack vectors and verify our findings. During this phase, we focused on:

- ensuring all key operations like vault creation, deposits, withdrawals, rebalancing, etc. were functioning correctly;
- ensuring the program correctly handled all edge cases in automated liquidity management;
- ensuring there were no possible access violations in vault administration;
- ensuring the protocol behaved fairly in fee distribution and yield calculations;
- ensuring external program dependencies and CPI calls were secure and properly validated;
- ensuring the arithmetic calculations for position management and kToken pricing were performed correctly; and
- ensuring there were no areas where the protocol could be misused or manipulated.

For fuzz testing, we used the Trident fuzzing framework designed for Solana programs written using the Anchor framework. We used fuzzing flows to test instruction sequences and invariants to verify correct state transitions after every successful transaction execution.

In the final phase, we compiled our findings and wrote this report. We also analyzed more complex attack vectors, particularly those involving interactions with external protocols. We discussed cross-protocol vulnerabilities and considered how attackers might exploit the protocol through its broader ecosystem integrations.

Our review resulted in 5 findings, ranging from Info to Medium severity.

The M1 allows protocol admins to set the withdrawal fee high enough to prevent withdrawals. Even though this setting would drastically lower the protocol's trustworthiness, so the motivation is low. The impact is high due to funds being locked in the vaults until manual intervention from the protocol

admins.

Ackee Blockchain Security recommends Kamino:

- read and review the complete audit report; and
- address all identified issues.

See Report Revision 1.0 for the system overview, trust model and comprehensive fuzz testing description.

Revision 1.1

Kamino engaged Ackee Blockchain Security to perform a fix review of the findings from the previous revision.

The review was performed between September 26 and October 14, 2025 on the commit `a37f6de`[2].

From the reported 5 findings, one issue was fixed and remaining issues were acknowledged.

The M1 was fixed by adding bounds to both `withdrawal_penalty_lamports` (former `withdrawal_fee_lamports`) and `withdrawal_penalty_bps` (former `withdrawal_fee_bps`) fields. The bounds are 10,000 base units and 1,000 basis points (BPS), respectively. While the issue can be marked as fixed, the upper limit of 10,000 base units applies in absolute terms to all supported currencies. Thus, if the currency is cbBTC, the bounds would be 10,000 base units of cbBTC (ten thousand Satoshis), which is currently worth approximately $11 USD, meaning users would not be able to withdraw amounts smaller than $11 USD.

The I1 was acknowledged, with the following comment:

> *Will discuss with vault managers the desired behaviour. More*

> *fine grained permissioning is planned for the smart contracts. No change now.*
>
> — Kamino team

The I2 was acknowledged, with the following comment:

> *No immediate actions. Will fix in a followup.*
>
> — Kamino team

The I3 was acknowledged, with the following comment:

> *No immediate actions. Will fix in a followup.*
>
> — Kamino team

The I4 was acknowledged, with the following comment:

> *No immediate actions. Will fix in a followup.*
>
> — Kamino team

No new findings were reported.

[1] full commit hash: `0c5aa727adde924cf7d8bbbd2f87e583a8aa04b7`

[2] full commit hash: `a37f6de1b3f91d08f9f898397ad28332bf1dec0b`

4. Findings Summary

The following section summarizes findings we identified during our review. Unless overridden for purposes of readability, each finding contains:

- *Description*
- *Exploit scenario* (if severity is low or higher)
- *Recommendation*
- *Fix* (if applicable).

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	0	1	0	0	4	5

Table 2. Findings Count by Severity

Findings in detail:

Finding title	Severity	Reported	Status
M1: Lack of withdrawal fee bounds leads to potential fund locking	Medium	1.0	Fixed
I1: Potential RBAC breach can lead to undesired allocations	Info	1.0	Acknowledged
I2: Unused Source Code	Info	1.0	Acknowledged
I3: Unused Rent Accounts	Info	1.0	Acknowledged
I4: Rent passed as account	Info	1.0	Acknowledged

Table 3. Table of Findings

Report Revision 1.0

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Max Kupchenko	Auditor
Matej Hyčko	Auditor
Felipe Donato	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

The Kamino Vault is a Solana program that implements an automated yield-generating vault system built on top of the Kamino Lending protocol. The vault accepts user deposits of a specific token (e.g., USDC) and invests these funds across multiple Kamino lending reserves to generate yield for depositors.

Users deposit tokens into the vault and receive proportional shares representing their ownership. The vault automatically allocates deposited funds across up to 25 different Kamino lending reserves based on configurable allocation weights and caps. The invest instruction deposits tokens into lending reserves to earn interest, while users can withdraw their funds by burning shares to receive their proportional share of the vault's assets under management (AUM).

The vault uses a weighted allocation system where funds are distributed across reserves based on target allocation weights, subject to per-reserve token caps and an optional unallocated tokens cap. The system automatically rebalances allocations during invest operations to maintain

target ratios.

Trust Model

The Kamino Vault implements Role-Based Access Control (RBAC) to manage access to the various program mechanisms.

In the Kamino Vault program, users must trust that:

- the Solana program `upgrade_authority` will initialize the correct `GlobalConfig` with appropriate withdrawal penalty limits; and
- the underlying Kamino Lending protocol will operate fairly and maintain reserve health.

The Kamino Vault contains multiple administrative roles for vault management:

- Vault Admin
- Allocation Admin
- Global Admin

For these roles, users must trust that:

- the Vault Admin will set reasonable management fees (up to 10% annually) and performance fees (up to 100%) without exploiting depositors;
- the Vault Admin will configure appropriate minimum deposit/withdraw amounts and investment parameters that don't prevent legitimate usage;
- the Vault Admin will add only legitimate and healthy Kamino lending reserves to the allocation strategy;
- the Vault Admin will set fair withdrawal penalties (up to 10% or 10,000 base units) that don't unreasonably restrict user withdrawals;
- the Vault Admin will withdraw accumulated fees responsibly and may

choose to give up fees to benefit all vault participants;

- the Vault Admin will appoint trustworthy Allocation Admins and transfer admin rights through the pending admin mechanism responsibly;
- the Allocation Admin will rebalance vault funds across lending reserves fairly and in the best interest of depositors;
- the Allocation Admin will not manipulate allocation weights to favor specific reserves with malicious intent;
- the Global Admin will set system-wide withdrawal penalty minimums fairly across all vaults;
- the Global Admin will transfer global admin rights responsibly through the pending admin mechanism; and

Additionally, users must trust that:

- the program `upgrade_authority` will not introduce malicious code changes that could drain vault funds or manipulate fee structures; and
- the vault's share token accounting will remain accurate and proportional to the underlying assets under management.

Fuzzing

During the audit, manually-guided fuzz tests were developed to assess the protocol's correctness, security, and robustness. Fuzz test templates are generated from the IDL created by Anchor and then implemented based on user needs. Notably, Trident allows the specification of flows or invariant checks.

Flows are important for helping the fuzzer better cover valid instruction sequences. On the other hand, invariant checks allow for the detection of undesired changes made during instruction execution. When an instruction is successfully invoked, the user can specify multiple invariant checks to ensure

that the contents of the accounts were updated as expected during the execution.

Over the period of fuzzing, two common types of failures can occur, a panic during instruction execution or a failure of the specified invariant check. The former can happen, for instance, when an unchecked arithmetic overflow is detected, while the latter is triggered by behavior that is defined as undesired (e.g. undesired balance change).

The fuzz tests implemented multiple flows, detailed in Appendix B. These flows tested various protocol operations, such as:

- vault setup and initialization;
- deposit and withdrawal operations;
- investment and liquidity management;
- fee collection and distribution; and
- administrative operations and authority management.

Findings

The following section presents the list of findings discovered in this revision. For the complete list of all findings, Go back to Findings Summary

M1: Lack of withdrawal fee bounds leads to potential fund locking

Medium severity issue

Impact:	High	Likelihood:	Low
Target:	programs/kvault/src/operations/vault_operations.rs	Type:	Logic error

Description

The `vault_admin_authority` can set multiple fields within the vault. Two of these fields are `withdrawal_fee_lamports` and `withdrawal_fee_bps`, which are unbounded. The following two code snippets show how the withdrawal fee is calculated:

Listing 1. Excerpt from vault_operations

```
let withdrawal_fee = common::get_withdrawal_fee(
    total_for_user,
    vault.withdrawal_fee_lamports,
    vault.withdrawal_fee_bps,
);
```

Listing 2. Excerpt from vault_operations

```
pub fn get_withdrawal_fee(
    total_amount_withdrawn: u64,
    withdrawal_fee_lamports: u64,
    withdrawal_fee_bps: u64,
) -> u64 {
    let withdrawal_fee_from_bps = Fraction::from(total_amount_withdrawn)
        .full_mul_int_ratio(withdrawal_fee_bps, FULL_BPS)
        .to_ceil::<u64>();
    withdrawal_fee_from_bps.max(withdrawal_fee_lamports)
}
```

The next step is to compare the calculated withdrawal fee with the amount the user should receive:

Listing 3. Excerpt from vault_operations

```
require!(
    withdrawal_fee < total_for_user,
    KaminoVaultError::WithdrawAmountLessThanWithdrawalFee
);
```

If the fee is greater than the amount the user should receive, the withdrawal fails and the user's funds are locked in the vault. This can be easily achieved by setting the withdrawal fee to 100% or setting the `withdrawal_fee_lamports` to a relatively high value.

Exploit scenario

Scenario 1

Bob, a protocol admin, can update the vault configuration.

Alice, a regular user, holds funds in the vault.

1. Bob sets the `withdrawal_fee_bps` to 100%.
2. Alice attempts to withdraw her funds.
3. Since the `withdrawal_fee_bps` is 100%, Alice's withdrawal fails and her funds remain locked in the vault.

Scenario 2

Bob, a protocol admin, can update the vault configuration.

Alice, a regular user, holds funds in the vault.

1. Bob sets the `withdrawal_fee_lamports` to a relatively high value.

2. Alice attempts to withdraw her funds.
3. Since the `withdrawal_fee_lamports` in absolute value exceeds the amount the user should receive, Alice's withdrawal fails and her funds remain locked in the vault.

Recommendation

Add bounds to the `withdrawal_fee_lamports` and `withdrawal_fee_bps` fields, and calculate the minimum of the `total_for_user` and `withdrawal_fee` to allow withdrawals even if the withdrawal fee exceeds the amount the user should receive.

Fix 1.1

The issue was fixed by adding bounds to both `withdrawal_penalty_lamports` (former `withdrawal_fee_lamports`) and `withdrawal_penalty_bps` (former `withdrawal_fee_bps`) fields. The bounds are 10,000 base units and 1,000 basis points (BPS), respectively. While the issue can be marked as fixed, the upper limit of 10,000 base units applies in absolute terms to all supported currencies. Thus, if the currency is cbBTC, the bounds would be 10,000 cbBTC (ten thousand Satoshis), which is currently worth approximately $11 USD, meaning users would not be able to withdraw amounts smaller than $11 USD.

Go back to Findings Summary

I1: Potential RBAC breach can lead to undesired allocations

Impact:	Info	Likelihood:	N/A
Target:	programs/kvault/src/state.rs, programs/kvault/src/handlers/handler_update_reserve_allocation.rs	Type:	Code quality

Description

The program allows two authorities to update vault allocations, the `vault_admin_authority` and the `allocation_admin`.

Listing 4. Excerpt from handler_update_reserve_allocation

```
let idx = vault.get_reserve_idx_in_allocation(&reserve_key);
// if the reserve is not already in allocation require the signer to be admin, otherwise the the allocation admin can also update the allocation
let is_vault_admin = ctx.accounts.signer.key() == vault.vault_admin_authority;
let is_allocation_admin = ctx.accounts.signer.key() == vault.allocation_admin;
match idx {
    Some(_) => {
        require!(
            is_allocation_admin || is_vault_admin,
            KaminoVaultError::WrongAdminOrAllocationAdmin
        );
    }
    None => {
        require!(
            is_vault_admin,
            KaminoVaultError::WrongAdminOrAllocationAdmin
        );
    }
}
let ctoken_vault_bump = ctx.bumps.ctoken_vault;
```

This means the `allocation_admin` can update the `target_allocation_weight` for a particular allocation.

The `vault_admin_authority` can additionally set the main `unallocated_weight` for the whole vault.

The issue occurs because the `allocation_admin` can set all allocations to 0, which would effectively override the `vault_admin_authority` setting. This is due to the following code:

Listing 5. Excerpt from state

```
pub fn refresh_target_allocations(&mut self, invested: &Invested) -> Result<()> {
    let total_tokens = self.compute_aum(&invested.total)?;
    let total_weight = self
        .vault_allocation_strategy
        .iter()
        .filter(|r| r.reserve != Pubkey::default() && r.token_allocation_cap > 0)
        .map(|r| r.target_allocation_weight)
        .sum::<u64>(); // this doesn't contain the unallocated weight, the amount to remain unallocated is computed first and then allocate to the reserves

    let mut remaining_tokens_to_allocate = total_tokens;
    let mut token_target_allocations = [Fraction::ZERO; MAX_RESERVES];

    // First handle unallocated amount if there's unallocated weight
    if self.unallocated_weight > 0 {
        let unallocated_cap = if self.unallocated_tokens_cap == 0 {
            u64::MAX
        } else {
            self.unallocated_tokens_cap
        };

        let unallocated_target = total_tokens.mul_int_ratio(
            self.unallocated_weight,
            total_weight + self.unallocated_weight,
        );
        let unallocated_tokens_target = unallocated_target.min(Fraction::from(unallocated_cap));
```

```
        remaining_tokens_to_allocate -= unallocated_tokens_target;
    }

    let mut remaining_weight_to_allocate = total_weight;
```

If all allocations are 0, the `unallocated_weight` is not used because setting all allocation weights to 0 will effectively override the `vault_admin_authority` setting, even if the `unallocated_weight` is set to 10%.

Recommendation

Prevent the scenario described above. Ensure that the `target_allocation_weight` for all allocations cannot result in allocations smaller than the `unallocated_weight`.

Mitigation is already partially in place, as the `vault_admin_authority` can update the `allocation_admin`, meaning if any admin behaves maliciously, the `vault_admin_authority` can update the `allocation_admin` to a different account.

Acknowledgment 1.1

The client responded with the following comment.

> *Will discuss with vault managers the desired behaviour. More fine grained permissioning is planned for the smart contracts. No change now.*
>
> — Kamino team

Go back to Findings Summary

I2: Unused Source Code

Impact:	Info	Likelihood:	N/A
Target:	programs/kvault/src/lib.rs	Type:	Code quality

Description

Type `KaminoVaultResult` is not used in the code.

Listing 6. Excerpt from lib

```
pub type KaminoVaultResult<T = ()> = std::result::Result<T,
KaminoVaultError>;
```

Recommendation

Remove the unused source code.

Acknowledgment 1.1

The client responded with the following comment.

> *No immediate actions. Will fix in a followup.*
>
> — Kamino team

Go back to Findings Summary

I3: Unused Rent Accounts

Impact:	Info	Likelihood:	N/A
Target:	programs/kvault/src/handlers/handler_update_reserve_allocation.rs, src/handlers/handler_init_vault.rs, src/handlers/handler_invest.rs	Type:	Code quality

Description

The `InitVault` and `UpdateReserveAllocation` contexts require `pub rent: Sysvar<'info, Rent>`, which is not used in the instructions.

Recommendation

Remove the unused accounts.

Acknowledgment 1.1

The client responded with the following comment.

> *No immediate actions. Will fix in a followup.*
>
> — Kamino team

Go back to Findings Summary

I4: Rent passed as account

Impact:	Info	Likelihood:	N/A
Target:	programs/kvault/src/handlers/handler_initialize_shares_metadata.rs	Type:	Code quality

Description

The `InitializeSharesMetadata` context requires `pub rent: Sysvar<'info, Rent>`, which is passed as an account.

Recommendation

Use `Rent::get()?` instead.

Acknowledgment 1.1

The client responded with the following comment.

> *No immediate actions. Will fix in a followup.*
>
> — Kamino team

Go back to Findings Summary

Report Revision 1.1

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Max Kupchenko	Auditor
Matej Hyčko	Auditor
Felipe Donato	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

Overview

Since there were no comprehensive changes in this revision, the complete overview is listed in the Executive Summary section Revision 1.1.

Appendix A: How to cite

Please cite this document as:

Ackee Blockchain Security, Audit Report | Kamino: Kamino Vaults, 14.10.2025.

Appendix B: Trident Findings

This section lists the outputs from the Trident framework used for fuzz testing during the audit.

B.1. Fuzzing

The following table lists all implemented execution flows in the Trident fuzzing framework.

ID	Flow	Added
F1	Setup: Initialize lending market, global config, reserve, and vault with initial configuration	1.0
F2	Regular vault operation: deposit → invest → withdraw	1.0
F3	Direct withdrawal: deposit → withdraw from available	1.0
F4	Multiple deposits: deposits → invest	1.0
F5	Fee management: deposit → invest → withdraw → fee operations cycle	1.0
F6	Multiple deposits and invests: mixed deposits and invests → withdraw	1.0
F7	Comprehensive vault operations: 10+ step complex flow with all operations	1.0
F8	Configuration management: update vault config parameters	1.0
F9	Metadata management: update shares metadata	1.0
F10	Admin management: set pending admin → update admin authority	1.0
F11	Borrowing flow: deposit reserve liquidity → init obligation → deposit collateral → borrow liquidity → repay liquidity	1.0

ID	Flow	Added
F12	Filler: Single deposit operation	1.0
F13	Filler: Single invest operation	1.0
F14	Filler: Single withdraw operation	1.0
F15	Filler: Single withdraw from available operation	1.0
F16	Filler: Single withdraw pending fees operation	1.0
F17	Filler: Single give up pending fees operation	1.0
F18	Filler: Single update reserve allocation operation	1.0

Table 4. Trident fuzzing flows

The following table lists all implemented invariant checks in the Trident fuzzing framework.

ID	Invariant	Added	Status
IV1	Deposit operations must preserve token conservation	1.0	Success
IV2	Deposit operations must issue correct shares	1.0	Success
IV3	Deposit operations must respect maximum limit	1.0	Success
IV4	Deposit operations must properly allocate funds	1.0	Success
IV5	Investment operations must maintain bidirectional consistency	1.0	Success
IV6	Investment operations must update vault availability	1.0	Success
IV7	Withdrawal operations must burn shares correctly	1.0	Success

ID	Invariant	Added	Status
IV8	Withdrawal operations must provide token compensation	1.0	Success
IV9	Withdrawal operations must maintain positive balances	1.0	Success
IV10	Fee withdrawal operations must compensate admin	1.0	Success
IV11	Fee withdrawal operations must preserve system token conservation	1.0	Success
IV12	Administrative operations must enforce authority restrictions	1.0	Success
IV13	Only pending admin can accept authority transfer	1.0	Success

Table 5. Trident fuzzing invariants

ackee
blockchain security

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz